

QUESTION 11(a)

Question:

To Store Commands, Maintain History Buffer, Handle Capacity Limits, Retrieve Entries, Display History, Validate Consistency.

```
#include <stdio.h>
#include <string.h>

#define MAX_HISTORY 5
#define MAX_LENGTH 100

char history[MAX_HISTORY][MAX_LENGTH];
int count = 0;

/* Add command to history */
void add_command(char command[])
{
    if (count < MAX_HISTORY)
    {
        strcpy(history[count], command);
        count++;
    }
    else
    {
        /* Remove oldest command */
        for (int i = 1; i < MAX_HISTORY; i++)
        {
            strcpy(history[i - 1], history[i]);
        }

        strcpy(history[MAX_HISTORY - 1], command);
    }
}

/* Display history */
void display_history()
{
    printf("\n==== Command History =====\n");

    if (count == 0)
    {
        printf("History is empty.\n");
    }
}
```

```
char command[MAX_LENGTH];
int position;

printf("==== Command History System =====\n");
printf("Maximum history size: %d\n", MAX_HISTORY);

while (1)
{
    printf("\nEnter command");
    printf(" (type 'history', 'get', or 'exit'): ");

    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    /* Exit */
    if (strcmp(command, "exit") == 0)
    {
        break;
    }

    /* Display history */
    if (strcmp(command, "history") == 0)
    {
        display_history();
        continue;
    }

    /* Retrieve command */
    if (strcmp(command, "get") == 0)
    {
        printf("Enter history number: ");
        scanf("%d", &position);
        getchar();

        retrieve_command(position);
        continue;
    }
}
```

```

/* Ignore empty commands */
if (strlen(command) == 0)
{
    printf("Empty command ignored.\n");
    continue;
}

/* Store command */
add_command(command);

printf("Command stored successfully.\n");
}

printf("\nHistory consistency check:\n");

if (count <= MAX_HISTORY)
    printf("Valid: History is within capacity.\n");
else
    printf("Invalid: History exceeds capacity.\n");

printf("Program terminated.\n");

return 0;

```

Output

```

sri_dhruthi@Dhruthi:~$ nano skilll1a.c
sri_dhruthi@Dhruthi:~$ gcc skilll1a.c -o skilll1a
sri_dhruthi@Dhruthi:~$ ./skilll1a
==== Command History System ====
Maximum history size: 5

Enter command (type 'history', 'get', or 'exit'): ls
Command stored successfully.

Enter command (type 'history', 'get', or 'exit'): pwd
Command stored successfully.

Enter command (type 'history', 'get', or 'exit'): date
Command stored successfully.

Enter command (type 'history', 'get', or 'exit'): whoami
Command stored successfully.

Enter command (type 'history', 'get', or 'exit'): uname
Command stored successfully.

Enter command (type 'history', 'get', or 'exit'): history
==== Command History ====
1: ls
2: pwd
3: date
4: whoami
5: uname

Enter command (type 'history', 'get', or 'exit'): exit

History consistency check:
Valid: History is within capacity.
Program terminated.

```

QUESTION 11(b)

Question:

To Create Pipeline Structures, Store Commands, Maintain Execution Order, Connect Processes, Validate Pipeline Layout, Test Configurations.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_COMMANDS 10
#define MAX_LENGTH 100

struct Pipeline
{
    char commands[MAX_COMMANDS][MAX_LENGTH];
    int count;
};

/* Add command to pipeline */
void add_command(struct Pipeline *pipeline, char command[])
{
    if (pipeline->count >= MAX_COMMANDS)
    {
        printf("Pipeline capacity exceeded.\n");
        return;
    }

    strcpy(pipeline->commands[pipeline->count],
           command);

    pipeline->count++;
}

/* Display pipeline */
void display_pipeline(struct Pipeline *pipeline)
{
    printf("\n==== Pipeline Structure =====\n");

    for (int i = 0; i < pipeline->count; i++)
    {

```

```

int main()
{
    struct Pipeline pipeline;
    pipeline.count = 0;

    char input[MAX_LENGTH];

    printf("==== Pipeline Structure =====\n");

    printf("Enter commands one by one.\n");
    printf("Type 'done' when finished.\n\n");

    while (pipeline.count < MAX_COMMANDS)
    {
        printf("Command %d: ",
               pipeline.count + 1);

        fgets(input, sizeof(input), stdin);
        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "done") == 0)
            break;

        if (strlen(input) == 0)
        {
            printf("Empty command not allowed.\n");
            continue;
        }

        add_command(&pipeline, input);
    }
}

```

```

/* Validate pipeline */
if (!validate_pipeline(&pipeline))
{
    printf("Invalid pipeline configuration.\n");
    return 1;
}

display_pipeline(&pipeline);

printf("\nPipeline contains %d commands.\n",
       pipeline.count);

printf("Execution order:\n");

for (int i = 0; i < pipeline.count; i++)
{
    printf("%d. %s\n",
           i + 1,
           pipeline.commands[i]);
}

/* Demonstrate process creation
   for each pipeline command.
*/

printf("\nCreating processes...\n");

for (int i = 0; i < pipeline.count; i++)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
    }
}

```

```

    if (pid == 0)
    {
        printf("Child process %d created for: %s\n",
               getpid(),
               pipeline.commands[i]);

        exit(0);
    }

/* Parent waits for all children */
for (int i = 0; i < pipeline.count; i++)
{
    wait(NULL);
}

printf("\nAll pipeline processes completed.\n");

return 0;
}

```

Output

```

sri_dhruthi@Dhruthi:~$ nano skill11b.c
sri_dhruthi@Dhruthi:~$ gcc skill11b.c -o skill11b
sri_dhruthi@Dhruthi:~$ ./skill11b
===== Pipeline Structure =====
Enter commands one by one.
Type 'done' when finished.

Command 1: ls
Command 2: grep txt
Command 3: wc -l
Command 4: done

===== Pipeline Structure =====
[1] ls ->
[2] grep txt ->
[3] wc -l

Pipeline contains 3 commands.
Execution order:
1. ls
2. grep txt
3. wc -l

Creating processes...
Child process 1955 created for: ls
Child process 1956 created for: grep txt
Child process 1957 created for: wc -l

All pipeline processes completed.
sri_dhruthi@Dhruthi:~$ |

```